

Document number: P1716R0
Date: 2019-06-15
Audience: Library Evolution Working Group
Reply-to: Tomasz Kamiński <tomaszkam at gmail dot com>

ranges compare algorithm are over-constrained

1. Introduction

In this paper, we argue that the ranges version of the compare algorithm (like `equal`, `mismatch`, `search`) are over-constrained (they impose the validity of invocations that are never used). Thus they compare limited functionality in comparison to their STL1 counterparts.

To address above issues we propose to replace the `IndirectRelation<T, U>` concept with the `IndirectBinaryPredicate<T, U>`, that requires only that `pred(*t, *u)` is well-formed (and drop unnecessary `pred(*t, *t)`, `pred(*u, *t)` and `pred(*u, *u)` requirements).

Changes proposed in this paper need to be considered in the C++20 timeline, as they would constitute breaking change after the publication of standard in the current form.

2. Revision history

2.1. Revision 0

Initial revision.

3. Motivation and Scope

With the current specification the range version of the compare algorithm (like `equal`, `mismatch`, `search`), are requiring that the provided functor `f` models the `IndirectRelation<T, U>` — that implies that given the pair of iterator `t` of type `T` and `u` of type `U`, the following expression needs to be well-formed:

- `f(*t, *t)`,
- `f(*t, *u)`,
- `f(*u, *t)`,
- `f(*u, *u)`.

[Note: For simplicity we ignore the projection functionality.]

Out of the above only the second (`f(*t, *u)`) expression is required for the implementation of the algorithms.

As consequence of the above decision, the code that was working correctly with the non-range version of the algorithm:

```
std::vector<std::string> texts;
std::vector<std::size_t> lengths;

auto [ti, si] = std::mismatch(texts.begin(), texts.end(),
                             lengths.begin(), lengths.end(),
                             [](std::string const& text, std::size_t length) { return text.size() == length; });
```

will not longer work, when migrated to ranges version:

```
auto [ti, si] = std::ranges::mismatch(text, lengths, [](std::string const& text, std::size_t length) { return text.size() == length; });
```

The other example of the limitation of the expressiveness of the current specification is the [issue reported by Eric Niebler on the github page](#), that lead to the creation of this paper.

The range-v3 calendar example has the following:

```
mismatch(rng-of-iterators, rng-of-sentinels, std::not_equal_to<>{})
```

This compiles in master but not in the deep-integration branch where `mismatch` is (properly) constrained with `IndirectRelation`.

To address the above issue we propose to replace `IndirectRelation` constrain with the `IndirectBinaryPredicate` (that will impose only `f(*t, *t)`) in the following algorithms:

- `find`,
- `find_first_of`,
- `adjacent_find`,
- `count`,
- `mismatch`,
- `replace` and `replace_copy`,
- `remove` and `remove_copy`.

In addition we propose to change `IndirectlyComparable` to be refinement of `IndirectBinaryPredicate` instead of `IndirectRelation`, thus addressing:

- `split_view`,
- `find_end`,
- `equal`,
- `search`,
- `search_n`.

Finally, we introduce the `EquivalenceRelation` (and `IndirectEquivalenceRelation`) to correctly constrain following algorithms (thier impose this requirement in non-range version):

- `unique` and `unique_copy`,
- `is_permutation`.

3.2. BinaryPredicate vs EquivalenceRelation

Above we propose to replace `IndirectRelation` with either `IndirectBinaryPredicate` or `IndirectEquivalenceRelation`, depending on the algorithm. This may lead to the question, if we should not use `IndirectEquivalenceRelation` for each of the algorithms — the predicates presented in examples from the above section, seem to model equivalence, but only lack overloads for other argument types combinations.

Such approach will however silently prevent the some frequent usages of the compare algorithms like:

```
// searching for first inversion in range
auto it = std::ranges::adjacent_find(vec, std::greater{});

auto searchPattern = [](std::string_view text, std::string_view pattern) { std::ranges::search(text, pattern); };
// checking if all patterns can be found with in corresponding corpus
std::ranges::equal(texts, patterns, searchPattern);
```

If these algorithms would be constrained with `IndirectEquivalenceRelation` the above code would still compile (all syntactic requirement as met as comparators are homogeneous), however, the code will have undefined behaviour — both `std::greater<>` and `searchPattern` does not meet the semantic requirements for equivalence relation, as they are not symmetric.

3.2. Specification requirements

Constrains currently placed on the compare algorithms are over-constrained in context of their specification. Majority of the algorithms are defined to find (or just inform about existence) a pair of iterators `i1` (pointing into first range) and `i2` (pointing into second range), for which the following expression:

```
invoke(pred, invoke(proj1, *i1), invoke(proj2, *i2))
```

returns either true or false value. [Note: Rest of them are defined in terms of other compare algorithms.]

To illustrate: the specification of `search` from [\[alg.search\]](#):

```
template<ForwardIterator I1, Sentinel<I1> S1, ForwardIterator I2,
        Sentinel<I2> S2, class Pred = ranges::equal_to,
        class Proj1 = identity, class Proj2 = identity>
requires IndirectlyComparable<I1, I2, Pred, Proj1, Proj2>
constexpr subrange<I1>
    ranges::search(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
                  Proj1 proj1 = {}, Proj2 proj2 = {});
template<ForwardRange R1, ForwardRange R2, class Pred = ranges::equal_to,
        class Proj1 = identity, class Proj2 = identity>
requires IndirectlyComparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr safe_subrange_t<R1>
    ranges::search(R1&& r1, R2&& r2, Pred pred = {},
                  Proj1 proj1 = {}, Proj2 proj2 = {});
```

Returns:

- `{i, i + (last2 - first2)}`, where `i` is the first iterator in the range `[first1, last1 - (last2 - first2))` such that for every non-negative integer `n` less than `last2 - first2` the condition
`bool(invoke(pred, invoke(proj1, *(i + n)), invoke(proj2, *(first2 + n))))`
is true
- Returns `{last1, last1}` if no such iterator exists.

Complexity: At most `(last1 - first1) * (last2 - first2)` applications of the corresponding predicate and projections.

To implement above specification only the invocation of `f(proj1(*i1), proj2(*i2))` is required, and any other combinations of argument (like `f(proj(*i1), proj1(*i1))`) is unnecessary.

3.3. Non-range specification

The old non-range algorithms are constrained using the `BinaryPredicate` requirements, that was implied by the template parameter. Per [\[algorithms.requirements\]](#), this only requires the predicate to be invocable with iterators provided in order of their occurrence in range signature:

When not otherwise constrained, the `BinaryPredicate` parameter is used whenever an algorithm expects a function object that when applied to the result of dereferencing two corresponding iterators or to dereferencing an iterator and type `T` when `T` is part of the signature returns a value testable as true. In other words, if an algorithm takes `BinaryPredicate` `binary_pred` as its argument and `first1` and `first2` as its iterator arguments with respective value types `T1` and `T2`, it should work correctly in the construct `binary_pred(*first1, *first2)` contextually converted to `bool` (`[conv]`).

[Note: `is_permutation`, `unique` and `unique_copy` imposes additional requirement on predicate: it should impose equivalence relation on the arguments.]

Due above, the non-range versions of the algorithms are more general and applicable, than their `ranges` counterpart. In addition, this unnecessary complicate the migration of the code to the rangified version of STL.

4. Proposed Wording

The proposed wording changes refer to N4810 (C++ Working Draft, 2019-03-15).

Apply following changes to [concept.syn] Header <concepts> synopsis:

```
// [concept.relation], concept Relation
template<class R, class T, class U>
    concept Relation = <em>see below</em>;

// [concept.equivalencerelation], concept EquivalenceRelation
template<class R, class T, class U>
    concept EquivalenceRelation = <em>see below</em>;

// [concept.strictweakorder], concept StrictWeakOrder
template<class R, class T, class U>
    concept StrictWeakOrder = <em>see below</em>;
```

Insert following section after [concept.relation] Concept EquivalenceRelation:

Concept EquivalenceRelation

[concept.equivalencerelation]

```
template<class R, class T, class U>
    concept EquivalenceRelation = Relation<R, T, U>;

A Relation satisfies EquivalenceRelation only if it imposes an equivalence relation on its arguments.
```

Apply following changes to section [iterator.synopsis] Header <iterator> synopsis:

```
template<class F, class I1, class I2 = I1>
    concept IndirectBinaryPredicateRelation = see below;

template<class F, class I1, class I2i = I1>
    concept IndirectEquivalenceRelation = see below;

template<class F, class I1, class I2 = I1>
    concept IndirectStrictWeakOrder = see below;
```

Apply following changes to section [indirectcallable.indirectinvocable] Indirect callable:

```
template<class F, class I1, class I2 = I1>
    concept IndirectBinaryPredicateRelation =
        Readable<I1> && Readable<I2> &&
        CopyConstructible<F> &&
        PredicateRelation<F&, iter_value_t<I1>&, iter_value_t<I2>&> &&
        PredicateRelation<F&, iter_value_t<I1>&, iter_reference_t<I2>>> &&
        PredicateRelation<F&, iter_reference_t<I1>, iter_value_t<I2>&> &&
        PredicateRelation<F&, iter_reference_t<I1>, iter_reference_t<I2>>> &&
        PredicateRelation<F&, iter_common_reference_t<I1>, iter_common_reference_t<I2>>>;

template<class F, class I1, class I2 = I1>
    concept IndirectEquivalenceRelation =
        Readable<I1> && Readable<I2> &&
        CopyConstructible<F> &&
        EquivalenceRelation<F&, iter_value_t<I1>&, iter_value_t<I2>&> &&
        EquivalenceRelation<F&, iter_value_t<I1>&, iter_reference_t<I2>>> &&
        EquivalenceRelation<F&, iter_reference_t<I1>, iter_value_t<I2>&> &&
        EquivalenceRelation<F&, iter_reference_t<I1>, iter_reference_t<I2>>> &&
        EquivalenceRelation<F&, iter_common_reference_t<I1>, iter_common_reference_t<I2>>>;

template<class F, class I1, class I2 = I1>
    concept IndirectStrictWeakOrder =
        Readable<I1> && Readable<I2> &&
        CopyConstructible<F> &&
        StrictWeakOrder<F&, iter_value_t<I1>&, iter_value_t<I2>&> &&
        StrictWeakOrder<F&, iter_value_t<I1>&, iter_reference_t<I2>>> &&
        StrictWeakOrder<F&, iter_reference_t<I1>, iter_value_t<I2>&> &&
        StrictWeakOrder<F&, iter_reference_t<I1>, iter_reference_t<I2>>> &&
        StrictWeakOrder<F&, iter_common_reference_t<I1>, iter_common_reference_t<I2>>>;
```

Apply following changes to section [alg.req.ind.cmp] Concept IndirectlyComparable:

```
template<class I1, class I2, class R, class P1 = identity,
        class P2 = identity>
    concept IndirectlyComparable
        = IndirectBinaryPredicateRelation<R, projected<I1, P1>, projected<I2, P2>>;
```

Apply following changes to section [algorithm.syn] Header <algorithm> synopsis:

```
namespace ranges {

template<InputIterator I, Sentinel<I> S, class T, class Proj = identity>
requires IndirectBinaryPredicateRelation<ranges::equal_to, projected<I, Proj>, const T*>
constexpr I find(I first, S last, const T& value, Proj proj = {});

template<InputRange R, class T, class Proj = identity>
requires IndirectBinaryPredicateRelation<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
constexpr safe_iterator_t<R>
    find(R&& r, const T& value, Proj proj = {});

template<InputIterator I, Sentinel<I> S, class Proj = identity,
        IndirectUnaryPredicate<projected<I, Proj>> Pred>
constexpr I find_if(I first, S last, Pred pred, Proj proj = {});

template<InputRange R, class Proj = identity,
        IndirectUnaryPredicate<projected<iterator_t<R>, Proj>> Pred>
constexpr safe_iterator_t<R>
    find_if(R&& r, Pred pred, Proj proj = {});

template<InputIterator I, Sentinel<I> S, class Proj = identity,
        IndirectUnaryPredicate<projected<I, Proj>> Pred>
constexpr I find_if_not(I first, S last, Pred pred, Proj proj = {});

template<InputRange R, class Proj = identity,
        IndirectUnaryPredicate<projected<iterator_t<R>, Proj>> Pred>
constexpr safe_iterator_t<R> find_if_not(R&& r, Pred pred, Proj proj = {});

}

[...]
```

```
namespace ranges {

template<InputIterator I1, Sentinel<I1> S1, ForwardIterator I2, Sentinel<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity,
        IndirectRelation<projected<I1, Proj1>,
        projected<I2, Proj2>> Pred = ranges::equal_to>
requires IndirectlyComparable<I1, I2, Pred, Proj1, Proj2>
constexpr I1 find_first_of(I1 first1, S1 last1, I2 first2, S2 last2,
        Pred pred = {},
        Proj1 proj1 = {}, Proj2 proj2 = {});

template<InputRange R1, ForwardRange R2, class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity,
        IndirectRelation<projected<iterator_t<R1>, Proj1>,
        projected<iterator_t<R2>, Proj2>> Pred = ranges::equal_to>
requires IndirectlyComparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr safe_iterator_t<R1>
    find_first_of(R1&& r1, R2&& r2,
        Pred pred = {},
        Proj1 proj1 = {}, Proj2 proj2 = {});

}

[...]
```

```
namespace ranges {

template<ForwardIterator I, Sentinel<I> S, class Proj = identity,
        IndirectBinaryPredicateRelation<projected<I, Proj>, projected<I, Proj>> Pred = ranges::equal_to>
constexpr I adjacent_find(I first, S last, Pred pred = {},
        Proj proj = {});

template<ForwardRange R, class Proj = identity,
        IndirectBinaryPredicateRelation<projected<iterator_t<R>, Proj>, projected<iterator_t<R>, Proj>> Pred = ranges::equal_to>
constexpr safe_iterator_t<R>
    adjacent_find(R&& r, Pred pred = {}, Proj proj = {});

}

[...]
```

```
namespace ranges {

template<InputIterator I, Sentinel<I> S, class T, class Proj = identity>
requires IndirectBinaryPredicateRelation<ranges::equal_to, projected<I, Proj>, const T*>
constexpr iter_difference_t<I>
    count(I first, S last, const T& value, Proj proj = {});

template<InputRange R, class T, class Proj = identity>
requires IndirectBinaryPredicateRelation<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
constexpr iter_difference_t<iterator_t<R>>
    count(R&& r, const T& value, Proj proj = {});

template<InputIterator I, Sentinel<I> S, class Proj = identity,
        IndirectUnaryPredicate<projected<I, Proj>> Pred>
constexpr iter_difference_t<I>
    count_if(I first, S last, Pred pred, Proj proj = {});

template<InputRange R, class Proj = identity,
        IndirectUnaryPredicate<projected<iterator_t<R>, Proj>> Pred>
constexpr iter_difference_t<iterator_t<R>>
```

```

        count_if(R&& r, Pred pred, Proj proj = {});
    }

    [...]

    namespace ranges {

    [...]

    template<InputIterator I1, Sentinel<I1> S1, InputIterator I2, Sentinel<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity,
        IndirectRelation<projected<I1, Proj1>,
            projected<I2, Proj2>> Pred = ranges::equal_to
        requires IndirectlyComparable<I1, I2, Pred, Proj1, Proj2>
    constexpr mismatch_result<I1, I2>
        mismatch(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
            Proj1 proj1 = {}, Proj2 proj2 = {});

    template<InputRange R1, InputRange R2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity,
        IndirectRelation<projected<iterator_t<R1>, Proj1>,
            projected<iterator_t<R2>, Proj2>> Pred = ranges::equal_to
        requires IndirectlyComparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
    constexpr mismatch_result<safe_iterator_t<R1>, safe_iterator_t<R2>>
        mismatch(R1&& r1, R2&& r2, Pred pred = {},
            Proj1 proj1 = {}, Proj2 proj2 = {});

    }

    [...]

    namespace ranges {

    template<ForwardIterator I1, Sentinel<I1> S1, ForwardIterator I2,
        Sentinel<I2> S2, class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>,
        IndirectEquivalenceRelation<projected<I1, Proj1>, projected<I2, Proj2>> Pred = ranges::equal_to
        requires IndirectlyComparable<I1, I2, Pred, Proj1, Proj2>
    constexpr bool is_permutation(I1 first1, S1 last1, I2 first2, S2 last2,
        Pred pred = {},
        Proj1 proj1 = {}, Proj2 proj2 = {});

    template<ForwardRange R1, ForwardRange R2, class Pred = ranges::equal_to,
        class Proj1 = identity, class Proj2 = identity>,
        IndirectEquivalenceRelation<projected<iterator_t<R1>, Proj1>, projected<iterator_t<R2>, Proj2>> Pred = ranges::equal_to
        requires IndirectlyComparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
    constexpr bool is_permutation(R1&& r1, R2&& r2, Pred pred = {},
        Proj1 proj1 = {}, Proj2 proj2 = {});

    }

    [...]

    namespace ranges {

    template<InputIterator I, Sentinel<I> S, class T1, class T2, class Proj = identity>
        requires Writable<I, const T2&> &&
        IndirectBinaryPredicateRelationBinaryPredicateRelation<ranges::equal_to, projected<I, Proj>, const T1*>
    constexpr I
        replace(I first, S last, const T1& old_value, const T2& new_value, Proj proj = {});

    template<InputRange R, class T1, class T2, class Proj = identity>
        requires Writable<iterator_t<R>, const T2&> &&
        IndirectBinaryPredicateRelationBinaryPredicateRelation<ranges::equal_to, projected<iterator_t<R>, Proj>, const T1*>
    constexpr safe_iterator_t<R>
        replace(R&& r, const T1& old_value, const T2& new_value, Proj proj = {});

    template<InputIterator I, Sentinel<I> S, class T, class Proj = identity,
        IndirectUnaryPredicate<projected<I, Proj>> Pred>
        requires Writable<I, const T&>
    constexpr I replace_if(I first, S last, Pred pred, const T& new_value, Proj proj = {});

    template<InputRange R, class T, class Proj = identity,
        IndirectUnaryPredicate<projected<iterator_t<R>, Proj>> Pred>
        requires Writable<iterator_t<R>, const T&>
    constexpr safe_iterator_t<R> replace_if(R&& r, Pred pred, const T& new_value, Proj proj = {});

    }

    [...]

    namespace ranges {

    template<class I, class O>
    using replace_copy_result = copy_result<I, O>;

    template<InputIterator I, Sentinel<I> S, class T1, class T2, OutputIterator<const T2&> O,
        class Proj = identity>
        requires IndirectlyCopyable<I, O> &&
        IndirectBinaryPredicateRelationBinaryPredicateRelation<ranges::equal_to, projected<I, Proj>, const T1*>
    constexpr replace_copy_result<I, O>
        replace_copy(I first, S last, O result, const T1& old_value, const T2& new_value,
            Proj proj = {});

    template<InputRange R, class T1, class T2, OutputIterator<const T2&> O,

```

```

        class Proj = identity>
requires IndirectlyCopyable<iterator_t<R>, 0> &&
        IndirectBinaryPredicateRelation<ranges::equal_to, projected<iterator_t<R>, Proj>, const T1*>
constexpr replace_copy_result<safe_iterator_t<R>, 0>
        replace_copy(R&& r, 0 result, const T1& old_value, const T2& new_value,
                Proj proj = {});

template<class I, class 0>
using replace_copy_if_result = copy_result<I, 0>;

template<InputIterator I, Sentinel<I> S, class T, OutputIterator<const T&> 0,
        class Proj = identity, IndirectUnaryPredicate<projected<I, Proj>> Pred>
requires IndirectlyCopyable<I, 0>
constexpr replace_copy_if_result<I, 0>
        replace_copy_if(I first, S last, 0 result, Pred pred, const T& new_value,
                Proj proj = {});

template<InputRange R, class T, OutputIterator<const T&> 0, class Proj = identity,
        IndirectUnaryPredicate<projected<iterator_t<R>, Proj>> Pred>
requires IndirectlyCopyable<iterator_t<R>, 0>
constexpr replace_copy_if_result<safe_iterator_t<R>, 0>
        replace_copy_if(R&& r, 0 result, Pred pred, const T& new_value,
                Proj proj = {});

}

[...]
```

```

namespace ranges {

template<Permutable I, Sentinel<I> S, class T, class Proj = identity>
requires IndirectBinaryPredicateRelation<ranges::equal_to, projected<I, Proj>, const T*>
constexpr I remove(I first, S last, const T& value, Proj proj = {});

template<ForwardRange R, class T, class Proj = identity>
requires Permutable<iterator_t<R>> &&
        IndirectBinaryPredicateRelation<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
constexpr safe_iterator_t<R>
        remove(R&& r, const T& value, Proj proj = {});

template<Permutable I, Sentinel<I> S, class Proj = identity,
        IndirectUnaryPredicate<projected<I, Proj>> Pred>
constexpr I remove_if(I first, S last, Pred pred, Proj proj = {});

template<ForwardRange R, class Proj = identity,
        IndirectUnaryPredicate<projected<iterator_t<R>, Proj>> Pred>
requires Permutable<iterator_t<R>>
constexpr safe_iterator_t<R>
        remove_if(R&& r, Pred pred, Proj proj = {});

}

[...]
```

```

namespace ranges {

template<class I, class 0>
using remove_copy_result = copy_result<I, 0>;

template<InputIterator I, Sentinel<I> S, WeaklyIncrementable 0, class T,
        class Proj = identity>
requires IndirectlyCopyable<I, 0> &&
        IndirectBinaryPredicateRelation<ranges::equal_to, projected<I, Proj>, const T*>
constexpr remove_copy_result<I, 0>
        remove_copy(I first, S last, 0 result, const T& value, Proj proj = {});

template<InputRange R, WeaklyIncrementable 0, class T, class Proj = identity>
requires IndirectlyCopyable<iterator_t<R>, 0> &&
        IndirectBinaryPredicateRelation<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
constexpr remove_copy_result<safe_iterator_t<R>, 0>
        remove_copy(R&& r, 0 result, const T& value, Proj proj = {});

template<class I, class 0>
using remove_copy_if_result = copy_result<I, 0>;

template<InputIterator I, Sentinel<I> S, WeaklyIncrementable 0,
        class Proj = identity, IndirectUnaryPredicate<projected<I, Proj>> Pred>
requires IndirectlyCopyable<I, 0>
constexpr remove_copy_if_result<I, 0>
        remove_copy_if(I first, S last, 0 result, Pred pred, Proj proj = {});

template<InputRange R, WeaklyIncrementable 0, class Proj = identity,
        IndirectUnaryPredicate<projected<iterator_t<R>, Proj>> Pred>
requires IndirectlyCopyable<iterator_t<R>, 0>
constexpr remove_copy_if_result<safe_iterator_t<R>, 0>
        remove_copy_if(R&& r, 0 result, Pred pred, Proj proj = {});

}

[...]
```

```

namespace ranges {

template<Permutable I, Sentinel<I> S, class Proj = identity,
        IndirectEquivalenceRelation<projected<I, Proj>> C = ranges::equal_to>
constexpr I unique(I first, S last, C comp = {}, Proj proj = {});

```

```

template<ForwardRange R, class Proj = identity,
        IndirectEquivalenceRelation<projected<iterator_t<R>, Proj>> C = ranges::equal_to>
requires Permutable<iterator_t<R>>
constexpr safe_iterator_t<R>
unique(R&& r, C comp = {}, Proj proj = {});

}

[...]

namespace ranges {

template<class I, class O>
using unique_copy_result = copy_result<I, O>;

template<InputIterator I, Sentinel<I> S, WeaklyIncrementable O,
        class Proj = identity, IndirectEquivalenceRelation<projected<I, Proj>> C = ranges::equal_to>
requires IndirectlyCopyable<I, O> &&
        (ForwardIterator<I> ||
         (InputIterator<O> && Same<iter_value_t<I>, iter_value_t<O>>) ||
         IndirectlyCopyableStorable<I, O>)
constexpr unique_copy_result<I, O>
unique_copy(I first, S last, O result, C comp = {}, Proj proj = {});

template<InputRange R, WeaklyIncrementable O, class Proj = identity,
        IndirectEquivalenceRelation<projected<iterator_t<R>, Proj>> C = ranges::equal_to>
requires IndirectlyCopyable<iterator_t<R>, O> &&
        (ForwardIterator<iterator_t<R>> ||
         (InputIterator<O> && Same<iter_value_t<iterator_t<R>>, iter_value_t<O>>) ||
         IndirectlyCopyableStorable<iterator_t<R>, O>)
constexpr unique_copy_result<safe_iterator_t<R>, O>
unique_copy(R&& r, O result, C comp = {}, Proj proj = {});

}

```

Apply following changes to section [alg.find] Find:

```

template<InputIterator I, Sentinel<I> S, class T, class Proj = identity>
requires IndirectBinaryPredicateRelation<ranges::equal_to, projected<I, Proj>, const T*>
constexpr I ranges::find(I first, S last, const T& value, Proj proj = {});

template<InputRange R, class T, class Proj = identity>
requires IndirectBinaryPredicateRelation<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
constexpr safe_iterator_t<R>
ranges::find(R&& r, const T& value, Proj proj = {});

```

Apply following changes to section [alg.find.first.of] Find first:

```

template<InputIterator I1, Sentinel<I1> S1, ForwardIterator I2, Sentinel<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity,
        IndirectRelation<projected<I1, Proj1>,
        projected<I2, Proj2>> Pred = ranges::equal_to>
requires IndirectlyComparable<I1, I2, Pred, Proj1, Proj2>
constexpr I1 ranges::find_first_of(I1 first1, S1 last1, I2 first2, S2 last2,
        Pred pred = {},
        Proj1 proj1 = {}, Proj2 proj2 = {});

template<InputRange R1, ForwardRange R2, class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity,
        IndirectRelation<projected<iterator_t<R1>, Proj1>,
        projected<iterator_t<R2>, Proj2>> Pred = ranges::equal_to>
requires IndirectlyComparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr safe_iterator_t<R1>
ranges::find_first_of(R1&& r1, R2&& r2,
        Pred pred = {},
        Proj1 proj1 = {}, Proj2 proj2 = {});

```

Apply following changes to section [alg.adjacent.find] Adjacent find:

```

template<ForwardIterator I, Sentinel<I> S, class Proj = identity,
        IndirectBinaryPredicateRelation<projected<I, Proj>, projected<I, Proj>> Pred = ranges::equal_to>
constexpr I ranges::adjacent_find(I first, S last, Pred pred = {},
        Proj proj = {});

template<ForwardRange R, class Proj = identity,
        IndirectBinaryPredicateRelation<projected<iterator_t<R>, Proj>, projected<iterator_t<R>, Proj>> Pred = ranges::equal_to>
constexpr safe_iterator_t<R>
ranges::adjacent_find(R&& r, Pred pred = {}, Proj proj = {});

```

Apply following changes to section [alg.is.permutation] Is permutation:

```

template<ForwardIterator I1, Sentinel<I1> S1, ForwardIterator I2,
        Sentinel<I2> S2, class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>,
        IndirectEquivalenceRelation<projected<I1, Proj1>, projected<I2, Proj2>> Pred = ranges::equal_to>
requires IndirectlyComparable<I1, I2, Pred, Proj1, Proj2>

```

```

constexpr bool ranges::is_permutation(I1 first1, S1 last1, I2 first2, S2 last2,
                                      Pred pred = {},
                                      Proj1 proj1 = {}, Proj2 proj2 = {});

template<ForwardRange R1, ForwardRange R2, class Pred = ranges::equal_to,
        class Proj1 = identity, class Proj2 = identity,
        IndirectEquivalenceRelation<projected<iterator_t<R1>, Proj1>, projected<iterator_t<R2>, Proj2>> Pred = ranges::equal_to
requires IndirectlyComparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr bool ranges::is_permutation(R1&& r1, R2&& r2, Pred pred = {},
                                      Proj1 proj1 = {}, Proj2 proj2 = {});

```

Apply following changes to section [alg.count] Count:

```

template<InputIterator I, Sentinel<I> S, class T, class Proj = identity>
requires IndirectBinaryPredicateRelation<ranges::equal_to, projected<I, Proj>, const T*>
constexpr iter_difference_t<I>
ranges::count(I first, S last, const T& value, Proj proj = {});

template<InputRange R, class T, class Proj = identity>
requires IndirectBinaryPredicateRelation<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
constexpr iter_difference_t<iterator_t<R>>
ranges::count(R&& r, const T& value, Proj proj = {});

```

Apply following changes to section [alg.mismatch] Mismatch:

```

template<InputIterator I1, Sentinel<I1> S1, InputIterator I2, Sentinel<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity,
IndirectRelation<projected<I1, Proj1>,
projected<I2, Proj2>> Pred = ranges::equal_to
requires IndirectlyComparable<I1, I2, Pred, Proj1, Proj2>
constexpr mismatch_result<I1, I2>
ranges::mismatch(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
                 Proj1 proj1 = {}, Proj2 proj2 = {});

template<InputRange R1, InputRange R2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity,
IndirectRelation<projected<iterator_t<R1>, Proj1>,
projected<iterator_t<R2>, Proj2>> Pred = ranges::equal_to
requires IndirectlyComparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr mismatch_result<safe_iterator_t<R1>, safe_iterator_t<R2>>
ranges::mismatch(R1&& r1, R2&& r2, Pred pred = {}, i
                 Proj1 proj1 = {}, Proj2 proj2 = {});

```

Apply following changes to section [alg.replace] Replace:

```

template<InputIterator I, Sentinel<I> S, class T1, class T2, class Proj = identity>
requires Writable<I, const T2&> &&
IndirectBinaryPredicateRelation<ranges::equal_to, projected<I, Proj>, const T1*>
constexpr I
ranges::replace(I first, S last, const T1& old_value, const T2& new_value, Proj proj = {});

template<InputRange R, class T1, class T2, class Proj = identity>
requires Writable<iterator_t<R>, const T2&> &&
IndirectBinaryPredicateRelation<ranges::equal_to, projected<iterator_t<R>, Proj>, const T1*>
constexpr safe_iterator_t<R>
ranges::replace(R&& r, const T1& old_value, const T2& new_value, Proj proj = {});

[...]

template<InputIterator I, Sentinel<I> S, class T1, class T2, OutputIterator<const T2&> O,
        class Proj = identity>
requires IndirectlyCopyable<I, O> &&
IndirectBinaryPredicateRelation<ranges::equal_to, projected<I, Proj>, const T1*>
constexpr replace_copy_result<I, O>
ranges::replace_copy(I first, S last, O result, const T1& old_value, const T2& new_value,
                    Proj proj = {});

template<InputRange R, class T1, class T2, OutputIterator<const T2&> O,
        class Proj = identity>
requires IndirectlyCopyable<iterator_t<R>, O> &&
IndirectBinaryPredicateRelation<ranges::equal_to, projected<iterator_t<R>, Proj>, const T1*>
constexpr replace_copy_result<safe_iterator_t<R>, O>
ranges::replace_copy(R&& r, O result, const T1& old_value, const T2& new_value,
                    Proj proj = {});

```

Apply following changes to section [alg.remove] Remove:

```

template<Permutable I, Sentinel<I> S, class T, class Proj = identity>
requires IndirectBinaryPredicateRelation<ranges::equal_to, projected<I, Proj>, const T*>
constexpr I ranges::remove(I first, S last, const T& value, Proj proj = {});

template<ForwardRange R, class T, class Proj = identity>
requires Permutable<iterator_t<R>> &&
IndirectBinaryPredicateRelation<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
constexpr safe_iterator_t<R>

```

```

    ranges::remove(R&& r, const T& value, Proj proj = {});

[...]

template<InputIterator I, Sentinel<I> S, WeaklyIncrementable O, class T,
        class Proj = identity>
    requires IndirectlyCopyable<I, O> &&
        IndirectBinaryPredicateRelation<ranges::equal_to, projected<I, Proj>, const T*>
    constexpr remove_copy_result<I, O>
        ranges::remove_copy(I first, S last, O result, const T& value, Proj proj = {});

template<InputRange R, WeaklyIncrementable O, class T, class Proj = identity>
    requires IndirectlyCopyable<iterator_t<R>, O> &&
        IndirectBinaryPredicateRelation<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
    constexpr remove_copy_result<safe_iterator_t<R>, O>
        ranges::remove_copy(R&& r, O result, const T& value, Proj proj = {});

```

Apply following changes to section [alg.unique] Unique:

```

template<Permutable I, Sentinel<I> S, class Proj = identity,
        IndirectEquivalenceRelation<projected<I, Proj>> C = ranges::equal_to>
    constexpr I ranges::unique(I first, S last, C comp = {}, Proj proj = {});

template<ForwardRange R, class Proj = identity,
        IndirectEquivalenceRelation<projected<iterator_t<R>, Proj>> C = ranges::equal_to>
    requires Permutable<iterator_t<R>>
    constexpr safe_iterator_t<R>
        ranges::unique(R&& r, C comp = {}, Proj proj = {});

[...]

template<InputIterator I, Sentinel<I> S, WeaklyIncrementable O,
        class Proj = identity, IndirectEquivalenceRelation<projected<I, Proj>> C = ranges::equal_to>
    requires IndirectlyCopyable<I, O> &&
        (ForwardIterator<I> ||
         (InputIterator<O> && Same<iter_value_t<I>, iter_value_t<O>>) ||
         IndirectlyCopyableStorable<I, O>)
    constexpr unique_copy_result<I, O>
        ranges::unique_copy(I first, S last, O result, C comp = {}, Proj proj = {});

template<InputRange R, WeaklyIncrementable O, class Proj = identity,
        IndirectEquivalenceRelation<projected<iterator_t<R>, Proj>> C = ranges::equal_to>
    requires IndirectlyCopyable<iterator_t<R>, O> &&
        (ForwardIterator<iterator_t<R>> ||
         (InputIterator<O> && Same<iter_value_t<iterator_t<R>>, iter_value_t<O>>) ||
         IndirectlyCopyableStorable<iterator_t<R>, O>)
    constexpr unique_copy_result<safe_iterator_t<R>, O>
        ranges::unique_copy(R&& r, O result, C comp = {}, Proj proj = {});

```

Update the value of the __cpp_ranges in table "Standard library feature-test macros" of [support.limits.general] to reflect the date of approval of this proposal.

5. Implementability

Implementation of the changes proposed in this paper, may be found in [following pull request](#) for [cmcstl2](#).

6. Acknowledgements

Special thanks and recognition goes to Sabre (<http://www.sabre.com>) for supporting the production of this proposal and author's participation in standardization committee.

7. References

1. Eric Niebler, "[Indirect]Relation + non-comparable sentinels and input iterators == :-(", (Issue 599, <https://github.com/ericniebler/stl2/issues/599>)
2. Richard Smith, "Working Draft, Standard for Programming Language C++" (N4810, <https://wg21.link/n4810>)
3. Casey Carter et al., "cmcstl2", (<https://github.com/CaseyCarter/cmcstl2>)
4. Tomasz Kamiński, "Implemented changes from P1716", (Pull request #310, <https://github.com/CaseyCarter/cmcstl2/pull/310>)